Session_15 activities worksheet

In this Session, we'll continue our survey of Monte Carlo computational methods with a look at autocorrelation and variational Monte Carlo applied to simple problems that are easily generalized.

A) Autocorrelation

When evaluating an average over Monte Carlo configurations, we want to skip the first n1 steps and then use data taken every n0 Monte Carlo steps. How do we determine how large to take n0 and n1? We'll use a simple integration problem to explore these issues, by integrating $x^2e^{-x^2}$ from 0 to 10 and dividing by the integral of e^{-x^2} (so this is like $< x^2 >$ with a probability distribution proportional to e^{-x^2}).

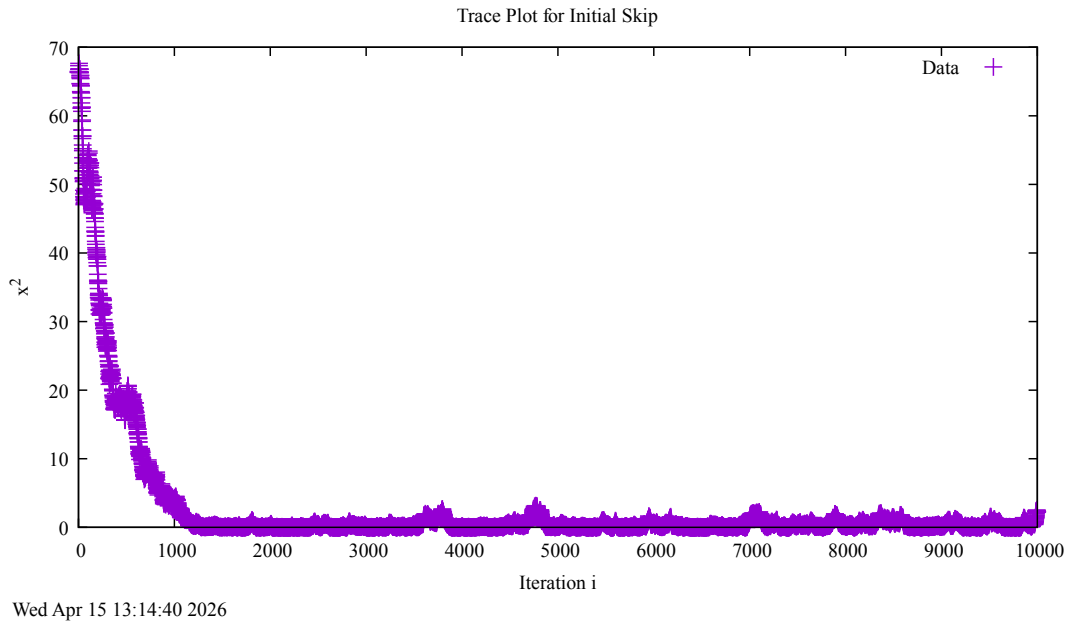
1. *Calculate the (normalized) integral in Mathematica for reference. Answer:*
   → Mathematica returned an integral of 0.5.
2. Consider first random sampling as a review/recap of our discussions.
   - Run the autocorrelation_test.cpp code a number of times (at least 10) and see how the random sampling estimates vary. *Is the calculation of errors in the code (e.g.* error_random*) consistent with the discussion in the background notes from Session_13? Explain.*
     →The calculation seems to be consistent with the discussion in Session 13, where the error relates to 1/sqrt(N) and averages like <x^2> and <x>^2.
   - *Are the estimated errors consistent with the actual deviations of the estimates from the exact result? (E.g., if the error is one standard deviation, about what fraction of the results should lie between the exact result +/- the error? What fraction do you observe?)*
     →The estimated errors are consistent with the actual deviations, as we expect the estimates to follow a normal distribution on the exact result. Hence, we should expect 68% of the results to lie within 1sigma, which we get that 6/10 trials did so (would be bettered with more trials).
   - Vary the number N of configurations used to make the estimates to see how the error scales. (The number of configurations is just the number of x's sampled in this case.) *With what power of N does it scale? Is this result consistent with expectations from the notes?*
     →We see that it roughly scales with 1/sqrt(N) (so multiplying N by 10 reduces error by 1/sqrt(10)). This is consistent with the expectations from the notes.
3. Next consider the Metropolis algorithm. There are three parameters for you to adjust (besides the total number of iterations): "max_step", "initial_skip" and "skip".
   - *What do each of these affect? (Why do we skip steps at the beginning? Why do we skip configurations? How do we know how much to change x? See the Session_15 notes for*

answers.)

→The initial_skip parameter affects the “burn-in” phase, skipping steps at the beginning to allow the MC to move to a high-probability region of the distribution. The skip parameter helps with statistical independence, skipping certain number of configurations between measurements to make sure samples are uncorrelated. Finally, max_step relates to the acceptance rate (we want around 50%) to explore enough of the configuration space.

- Adjust "max_step" so the acceptance rate is around 50% (or less). [If stepping a vector **X**, we test every component to form a Monte Carlo step and look for choosing a step size so that the acceptance rate is around 50%.] *What "max_step" did you use?*
  →A max_step around 1.02 seemed to float around 50% acceptance rate.

- Devise a way to decide on the "initial_skip" (use a plot). Set "initial_skip" in the code to this value. (Note that your result will depend on your choice of "max_step".) *Explain your value.*
  →What you can do is output the iteration and x^2 to a dat file. At initial iterations, you should see the starting value of x is far from the peak and it drifts towards “equilibrium” at a plateau. This depends on your max_step size, because a good max step could equilibrate without this drift period. Otherwise, you can find the start of the plateau to take that as the initial skip. An example of this drift with a bad max_step size is used below, where you would want an initial skip of about 1500. For our max_step, we find that the system is in this equilibrium very quickly, and an initial skip of about 100 is sufficient.



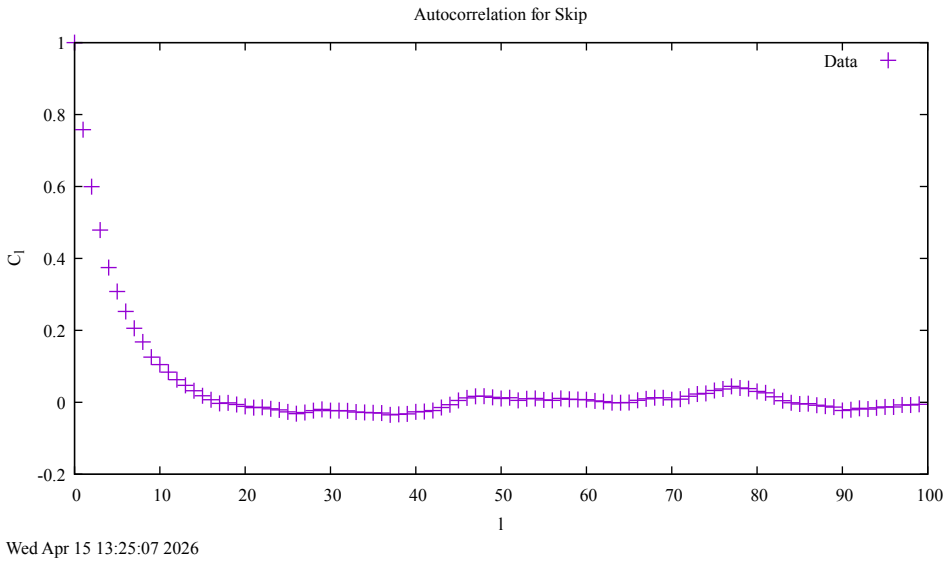
4. To figure out a good value for "skip", we'll calculate the "autocorrelation function". This is defined in the Session_15 notes as equation (15.2). For us, "A" is "x^2". Your task is to generate a figure like the one in the notes.
    - *What is the value of the autocorrelation function C(l) at l=0?*

      →It should be one (which it is).

    - *What is the condition that makes C(l) tend to zero (based on the defining equation)?*

      →The condition is that the covariance term vanishes, making C(l) go to zero (the covariance term being the <x_i x_i+l> - <x_i>^2 term, or the numerator).

    - You have everything you need in the code already for (15.2) *except* for the leftmost average. Modify the code to calculate that and then print out and plot C(l). Attach the plot. [Hint: An easy way (although not the most efficient way!) is to introduce an array "result_save[i]" in the Monte Carlo step loop and save all of the "result" evaluations. Then after this loop you can step through values of l (say from 0 to 100) and calculate (15.2) for each l (you should only use [iterations - l] configurations to do this calculation; why?). Then you can calculate (15.2) and output it.]
      →Below is the plot, which confirms our C(0)=1 and also is the same shape as the one in the notes. We can then pick a skip of about 20.



5. Now that the Metropolis results are reliable, repeat step 2. above, but now for the Metropolis sampling. *What do you find?*

   →Like random sampling, we get in general that roughly 68% of the time, our Metropolis answer falls within 1 sigma, and the error also scales as 1/sqrt(N). We also now find that the result from Metropolis is closer to the 0.5 expected value with smaller error bars than random sampling. Before, without the parameters tuned, we still got a close result, but we weren't always close to the result based on standard deviations.

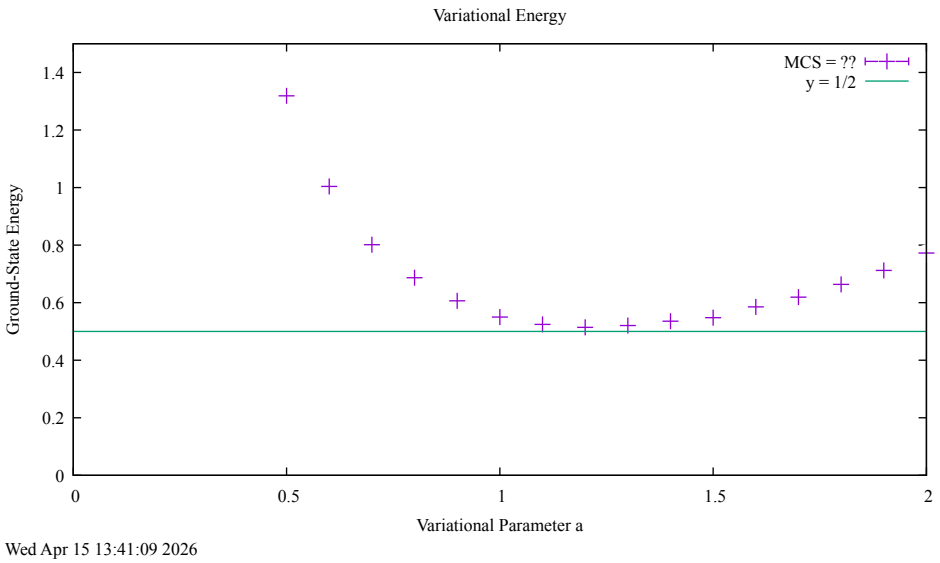
B) Variational Monte Carlo

We'll do a simple example of variational Monte Carlo to illustrate the basic idea. Generalizing to more complex systems is straightforward (but takes a lot longer to run!).

1. Take a look at the "variational_SHO.cpp" code and see how it implements variational Monte Carlo for a one-dimensional harmonic oscillator using the VariationalMC class. Compile, link, and run it (there is a makefile). *What is the trial wave function? Why is this a reasonable one to use?*
   →The trial wavefunction is sqrt(a/2) * 1/cosh(a*x). This is reasonable because it has the correct qualitative shape of an SHO, it is an even function that decayse for large |x|, and is analytically simple and variationally flexible.

2. You'll be asked to supply a range and step size for the variational parameter "a". This will require some experimentation to make sure the minimum with respect to "a" is in the interval you select.
3. The variational_SHO.dat file can be plotted in gnuplot. The plotfile "variational_SHO.plt" is provided to plot it with error bars. Try it out a few times each with 100, 1000, 10000 steps *(attach a plot with a sample result). Does the graph make sense for a variational calculation? What about the error bars? How might you modify the code to find the minimum automatically (rather than graphically)?*

   →Below is the plot. The graph does make sense, as it follows a convex "U" shape, characteristic of a variational search for a minimum. The error bars seem to shrink significantly with more MC steps, consistent with MC error scaling. Instead of doing this graphically, you could implement some feature that finds the parameter a, by using a GSL minimizer or by implementing gradient descent algorithms.

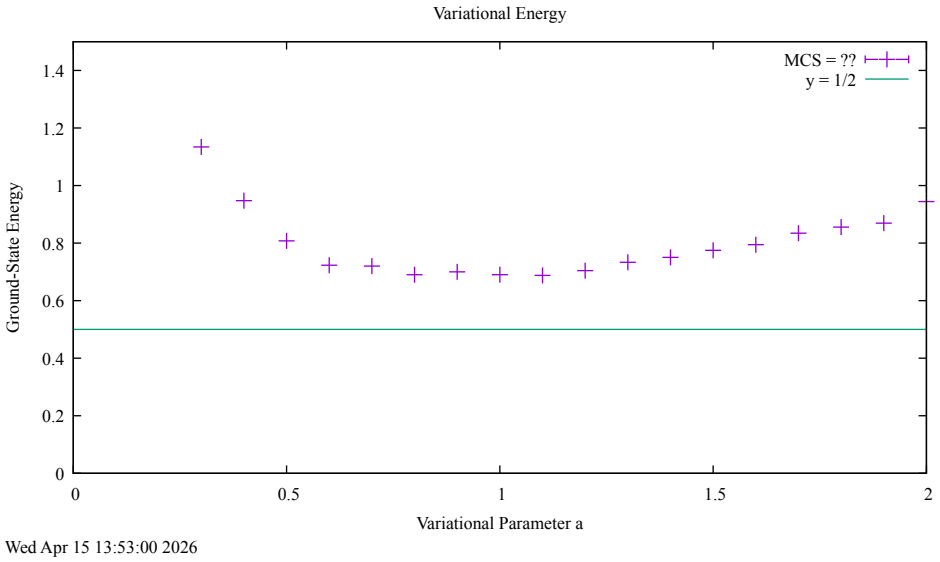


4. *Does this code implement all the features explored in the "Autocorrelation" section? If not, how would you improve this code?*

   →I don't think it implements all the features. One thing I noticed is that you can set the max step, but I don't think you can set an initial skip, which might be a good addition as well. Also, I did not find where the code even implements the autocorrelation calculation, and there is no introduced skip parameter either.

5. Bonus: The trial function is too good a guess. Try modifying it to another reasonable form with one variational parameter "a" (use your imagination!). You'll have to modify the "Psi" and "E_local" functions. *Find the minimum graphically and compare to the exact solution.*

   →Below is the plot using a Psi of 1/(1+a*x^2). We can see that this function still finds a minimum, but it's not as good as the "too good" trial function, and it's lowest energy is still higher than the exact solution.



C) Checking for Bad Input Values

You may have noticed that the nonlinear pendulum codes with the menus went crazy if you accidently typed a letter or a symbol rather than an integer. Here we see how to avoid this.

1. Use make_input_check to create input_check and run it. First try some integers and then -1 to see how it is supposed to work. Then restart it and put in a letter or symbol (or whatever you want). To stop it, use Ctrl-c.
2. A fix is given in input_check_fixed (which has its own makefile). Try it out. Look at the printout or on the screen to see what was done. *What was done to fix it?*
   →In the original check, the line cin>>x fails because it cannot parse a character into an int variable. In the fixed version, basically captures if the input is good. When doing

(bool)(cin>>x), the value is inputted and the bool checks the "health" of the input stream. If the health is bad (false value), you can then deal with the error accordingly and prevent failure by clearing the input stream and fetching the bad input.

D) Reflection

Write down which of the six learning goals on the syllabus this worksheet addressed for you and how:

→This worksheet helped me learn how to write correct, clear computer code. I learned more about physics through computational solutions, like the variational principle. I also learned more in-depth algorithms for computational physics, like deeper Metropolis algorithm understanding and the autocorrelation function.